

Malware Disassembly and Code Analysis Using Ghidra

EX. NO: 10

Date :

1. AIM

To set up a secure analysis environment and use the **Ghidra reverse engineering framework** to disassemble and decompile binary code, analyze low-level assembly instructions, examine strings and imported libraries, and interpret program execution flow.

2. SOFTWARE / TOOLS REQUIRED

Requirement	Details
Operating System	Isolated Virtual Machine with Windows/Linux OS
Forensics / Reverse Engineering Tool	Ghidra
Runtime Environment	Java Development Kit (JDK)
Sample Files	Controlled test binaries
Analysis Components	CodeBrowser, Decompiler, Function Graph

3. THEORY

Malware analysis involves examining compiled software to understand its structure, capabilities, and behavior. **Static analysis** allows a binary to be examined without executing it, making it useful for forensic investigation and reverse engineering. **Ghidra** is an open-source software reverse engineering framework developed by the National Security Agency (NSA). It provides tools for analyzing binary programs when the original source code is unavailable. The **CodeBrowser** component allows analysts to examine machine instructions and assembly code. The **Decompiler** converts low-level instructions into C-like pseudocode, making program logic easier to understand.

4. PROCEDURE

Step 1: Installation and Project Initialization

Open the Linux terminal and install Ghidra: `sudo apt update`

`sudo apt install ghidra -y`

Launch Ghidra: `ghidra`

Create a new project and select a suitable project name, such as: `Ghidra_Malware_Analysis`

Create a **non-shared project** for the analysis.

Step 2: Import the Binary

Navigate to: **File → Import File**

Select the controlled target binary and proceed with the default import configuration. After importing the binary, open it in **CodeBrowser**. Run the initial **Auto Analysis** when prompted.

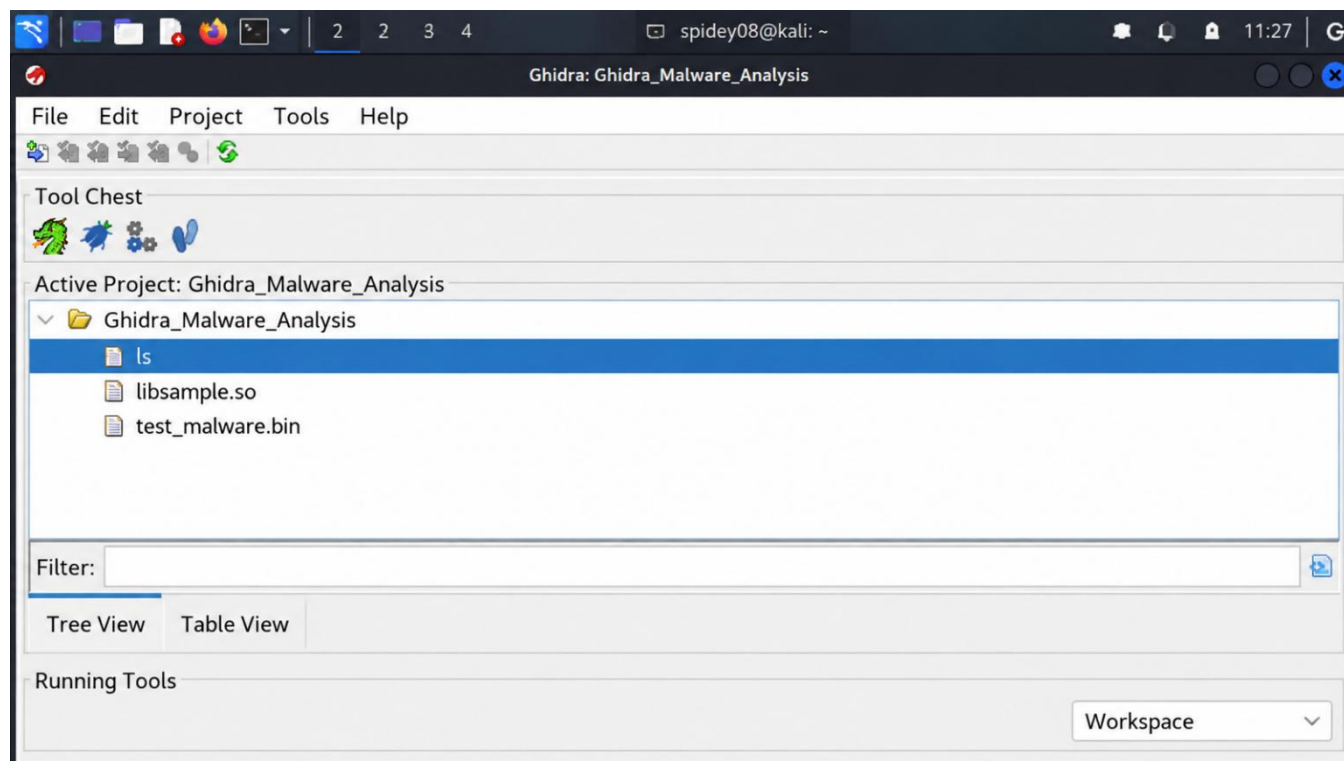


Figure 1: Initial Ghidra Project Window Showing the Imported Binary

Step 3: Examine the CodeBrowser and Disassembly

After Auto Analysis is completed, examine the CodeBrowser interface.

The Listing window displays the disassembled machine instructions and the associated program information.

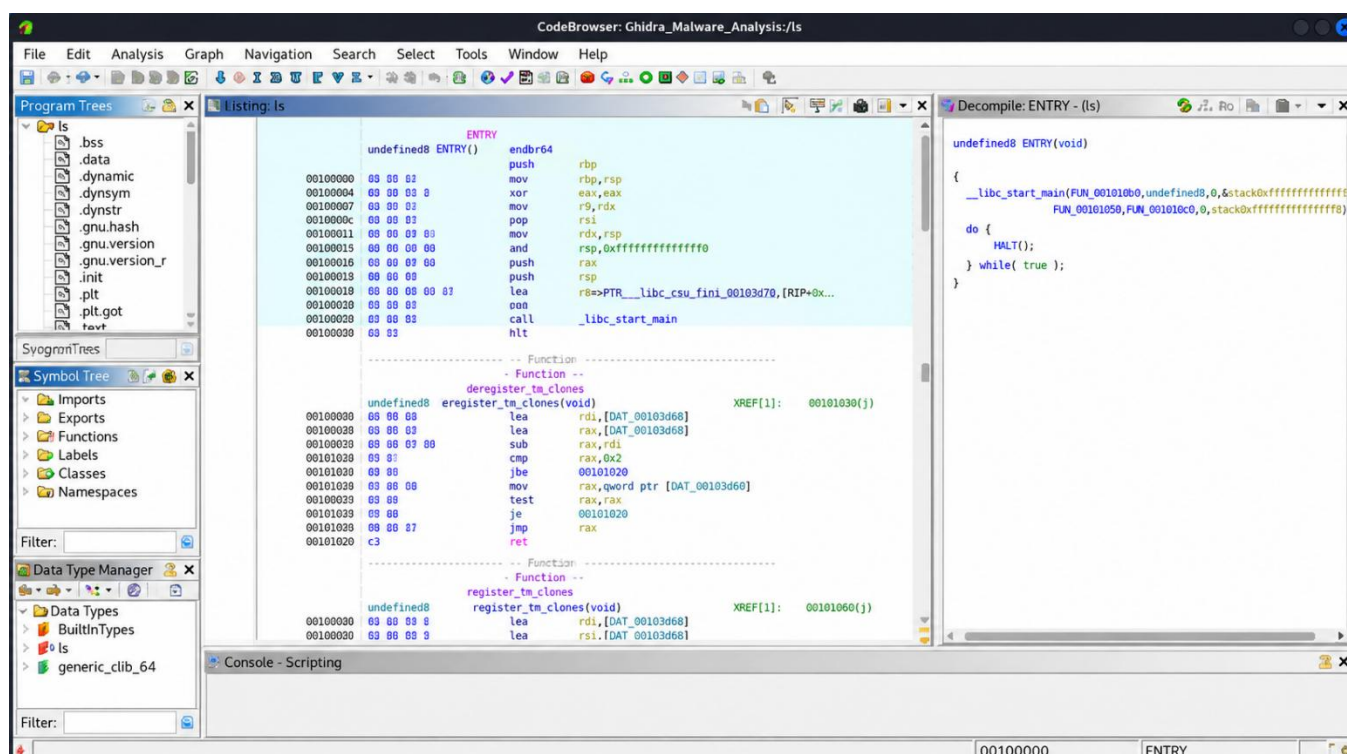


Figure 2: Ghidra CodeBrowser Interface Displaying the Disassembled Binary

Step 4: Extract Strings and Examine Imports

Navigate to: **Window** → **Defined Strings** Examine the strings identified within the binary. Next, use the **Symbol Tree** and expand the **Imports** section to identify external libraries and functions used by the program. For the analyzed Linux binary, system-library can be examined. Strings and imports can provide useful information about the functionality and dependencies of a binary.

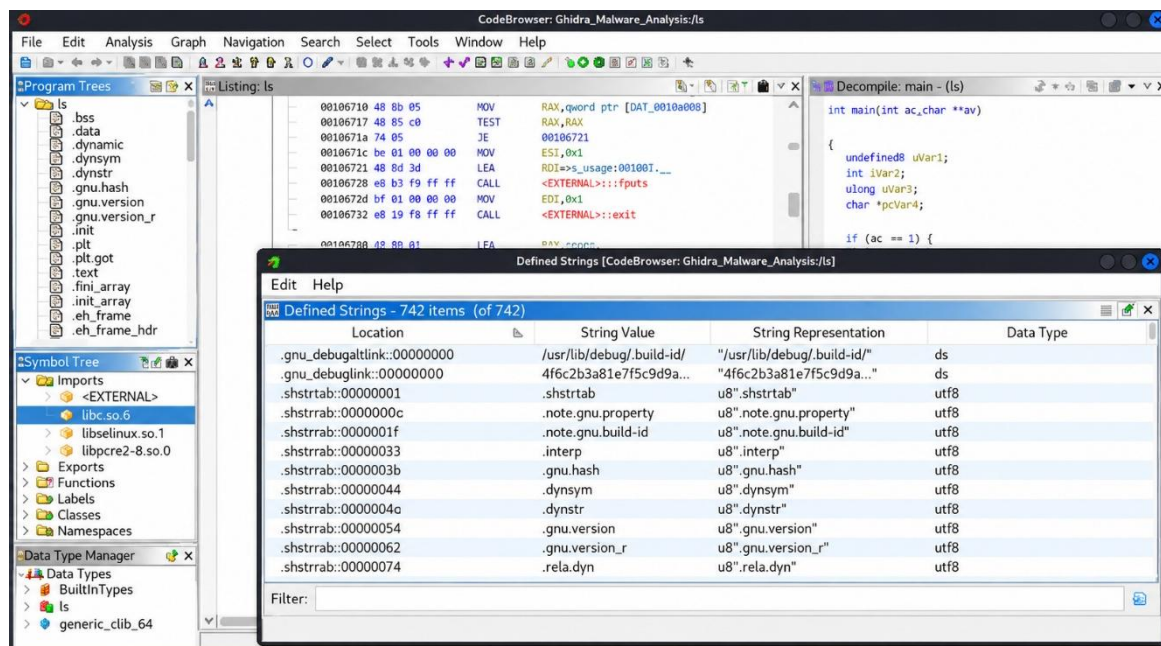


Figure 3: Defined Strings and Imported System Libraries

Step 5: Perform Function Decompile

In the **Symbol Tree**, expand the **Functions** section. Locate the entry function and select it. Examine the assembly instructions in the **Listing** window. The **Decompiler** window provides a C-like representation of the selected function. Compare the assembly instructions with the generated pseudocode to understand the function's logical structure.

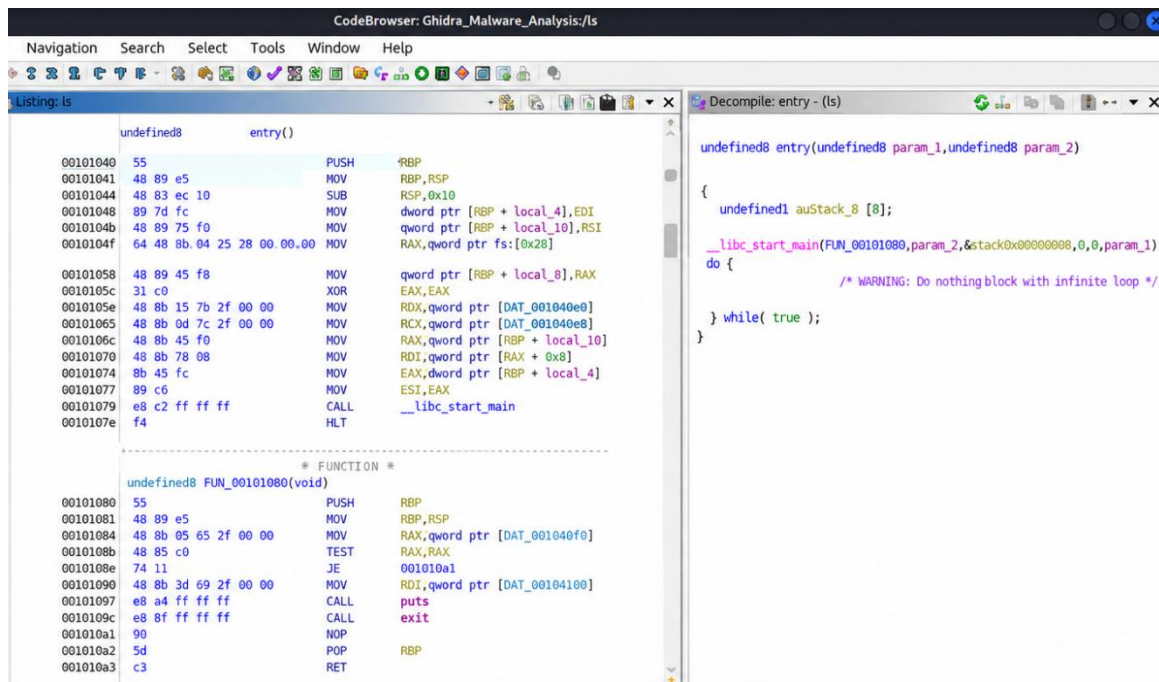


Figure 4: Decompiled C-Like Pseudocode of the Entry Function

Step 6: Identify the Main Execution Function

For a stripped binary, the original function names may not be available.

Examine the entry function and identify the function pointer. The referenced function can then be selected and examined to identify the main execution logic. For the analyzed binary, the relevant function can be followed through the reference provided in the Decompiler.

5. OBSERVATIONS / FORENSIC FINDINGS

Code Structure - Ghidra successfully analyzed the imported ELF binary and displayed its executable structure, instructions, data sections, and associated metadata.

Strings and Imports - Defined strings and imported libraries were identified from the binary. System-library dependencies such as libc.so.6 were examined as part of the static analysis.

Decompilation - The Ghidra Decompiler translated low-level assembly instructions into C-like pseudocode. This provided a more understandable representation of the program's execution logic.

Execution Flow - The Function Graph provided a visual representation of the function's control-flow structure, including branches and loops.

Stripped Binary Analysis - The analyzed binary did not expose all original developer-defined symbols. Function references were therefore examined through the program's entry-point execution flow and references associated with `__libc_start_main`.

6. LIMITATION

Static analysis provides information about the structure and potential behavior of a binary, but it does not by itself establish the complete runtime behavior of the program. A stripped or obfuscated binary may also make function names, variables, and program logic more difficult to interpret. Therefore, additional techniques such as controlled dynamic analysis may be required for comprehensive behavioral investigation.

7. CONCLUSION

Ghidra was successfully used to perform static analysis of the target binary. The experiment demonstrated the process of importing a binary, performing automatic analysis, examining disassembled instructions, extracting strings and imports, decompiling functions into C-like pseudocode, and analyzing program execution flow using the Function Graph. These techniques provide a structured approach for examining binary code during digital forensic and reverse-engineering investigations.

RUBRICS

Criteria & Marks Assigned	Mark Allotted	Mark Awarded
1. GitHub Activity & Submission Regularity	3	
2. Application of Forensic Tools & Practical Execution	3	
3. Documentation & Reporting	2	
4. Engagement, Problem-Solving & Team Collaboration	2	
Total	10	

8.RESULT

The target binary was successfully analyzed using **Ghidra**. Disassembly, string and import analysis, function decompilation, and control-flow analysis were performed to interpret the structure and execution logic of the binary.